Can Feature-Level Tests Predict LLM-Based C-to-Rust Translation Reliability?

Ariel E. Kellison, Laura Titolo, Ayon Basumallik, Niranjan Hasabnis, Cole Hunter, Chris Ladden, Tom Yotam
Code Metal, Boston MA, USA
{ariel.kellison,laura,ayon,niranjan,cole,chris.ladden,tom}@codemetal.ai

***Abstract*—Translating C codebases to Rust can provide stronger safety guarantees while retaining low-level control and performance. Although large language models (LLMs) can translate C programs into Rust, their support for individual C language features is not well characterized. This paper introduces a compiler-inspired, feature-level validation benchmark for C-to-Rust translation that isolates individual C language constructs using targeted tests derived systematically from the C11 standard. Across twelve evaluated LLMs, mean success rates range from 4.0% to 87.2%, and 16 of 46 features are covered by no more than half of the models. In a case study combining three features, a zero success rate on any constituent test coincides with a zero success rate on the combined program, providing preliminary evidence that feature-level tests may predict translation reliability.**

***Index Terms*—C-to-Rust translation, language conformance testing, LLMs**

I. Introduction

The translation of C programs to Rust is motivated by the need to evolve large, long-lived C codebases toward languages that provide stronger safety guarantees while preserving low-level control and performance. Existing C-to-Rust translation efforts have primarily relied on static analyses and rule-based rewriting [1], [2], [3], [4], [5], [6], [7]. These approaches offer predictability and explicit correctness criteria, but often have limited coverage and difficulty producing idiomatic Rust. Recent studies have evaluated large language models (LLMs) as an alternative for translating parts of real-world C programs into readable Rust [8], [9], [10], [11], [12], [13], [14], [15], [16].

However, unlike compiler-based translators, the behavior and limitations of LLM-based translation systems are not well characterized. While LLMs can succeed on many whole-program benchmarks, it remains unclear which parts of the C language they reliably support, and how robust that support is across syntactic and semantic variations. Moreover, because LLM-based translation does not proceed via explicit AST-to-AST rewriting or formal transformation rules, traditional compiler verification and validation techniques do not apply directly.

Whole-program translation correctness depends in part on correctly handling the individual language features a program exercises. In compiler validation, this observation motivates language conformance testing, in which implementations are evaluated against test suites derived from a language standard and designed to exercise individual features. In this paper, rather than treating the quality of LLM-based program translations as a purely whole-program property, we evaluate translation behavior at the level of individual C language features. This perspective complements existing whole-program evaluations, identifies failure modes that may remain hidden when multiple features interact, and, as we illustrate with a combined-program case study, provides preliminary evidence that feature-level results may indicate reliability on programs that combine several features.

We implement this approach as a feature-level validation framework for LLM-based C-to-Rust translation, with the following contributions.

Contribution 1: A compiler-inspired validation methodology for LLM-based translation. We adapt language conformance testing, a standard technique for validating compilers, to the evaluation of LLM-based program translation, treating each language feature as an independent unit of correctness.

Contribution 2: A C11 language feature benchmark (Section II). We construct a benchmark of 131 tests derived from the C11 standard, isolating 46 core language features. Additionally, we introduce *feature coverage* and *translation consistency* to characterize LLM-based translation quality. The benchmark and evaluation framework are publicly available: https://github.com/CodeMetalAI/c-to-rust-language-feature-bench.

Contribution 3: An empirical evaluation of twelve language models (Section III). We evaluate programming-specialized and general-purpose language models using the proposed benchmark. Our results show variation in feature support and reliability among LLMs, even among models with similar aggregate translation scores.

Contribution 4: Preliminary evidence that feature-level tests indicate translation reliability (Section IV). In a combined-program case study, a zero mean success rate on any constituent feature test coincides with a zero mean success rate on the combined program, while every model with a nonzero mean success rate on all constituent tests also has a nonzero mean success rate on the combined program.

II. Language Feature Support Benchmark

We introduce a feature-level benchmark for evaluating C-to-Rust translation systems. The benchmark isolates individual C language features using test cases derived from Clause 6 of the ISO/IEC 9899:2011 (C11) standard. We target C11 because it

https://doi.org/

This article is licensed under a Creative Commons Attribution 4.0 International License

```c
goto lab3; /* invalid */
{
  double a[n];
  a[j] = 4.4;
lab3:
  a[j] = 3.3;
  goto lab4; /* valid */
  a[j] = 5.5;
lab4:
  a[j] = 6.6;
}
goto lab4; /* invalid */
```

```c
int main(void) {
  int n = 4, j = 1;
  int saw_lab4 = 0;
  {
    double a[n];
    a[j] = 4.4;
    goto lab4;
/* valid: within VLA scope */
    a[j] = 5.5; /* skipped */
  lab4:
    saw_lab4 = 1;
    a[j] = 6.6;
  }
  if (!saw_lab4)
    return 1;
  return 0;
}
```

Fig. 1: Original illustrative fragment from Example 2 in Clause 6.8.6.1 (left). Derived positive test case retaining only the well-formed `goto` behavior (right).

is widely supported by `gcc` and `clang` and extends C99, the focus of prior work on static C-to-Rust translation [17].

A. Constructing the Benchmark

The benchmark is constructed by extracting and adapting examples from the C11 standard into *positive test cases*, a method used in compiler conformance testing [18], [19]. A positive test case is a program that is well-formed under the language rules and has fully defined behavior; it compiles and executes successfully while exercising a specific language feature. In addition to illustrative examples, tests are also derived from prose descriptions in subclauses that do not contain examples but specify concrete, testable language rules.

Examples in the C11 standard are primarily explanatory and may include both well-formed and ill-formed code. The benchmark, however, consists exclusively of tests derived from well-formed examples. When an example illustrates multiple behaviors, it may yield multiple tests. Figure 1 illustrates our extraction process using Example 2 from Clause 6.8.6.1, which concerns restrictions on `goto` statements involving variably modified types. The derived test retains only the valid `goto` behavior, ensuring that the resulting program is well-formed and exercises the intended feature.

The benchmark comprises 131 tests spanning the C11 core language. Each test targets one language feature, though a feature may have multiple tests covering distinct aspects. The median test is 26 lines of C, excluding comments to avoid influencing model behavior. Tests are not uniform across features, reflecting the standard's structure. Table I summarizes the distribution[1].

A natural question is why existing C conformance or compiler validation test suites are not reused. Comprehensive suites such as those from Plum Hall [20] support feature-level validation but are proprietary and unsuitable for an open benchmark. Public test suites from compilers such as `gcc` and `clang` are primarily designed for regression testing and contain relatively few systematic positive tests that isolate individual language features. We therefore construct an open benchmark derived directly from the C11 standard, preserving the spirit of conformance testing while remaining publicly available.

[1]Clauses 6.1, 6.6, and 6.11 are omitted because they contain no examples in the C11 standard, and we did not construct benchmark tests for them.

TABLE I: **Benchmark coverage of C11 Clause 6.** For each subclause, we report the number of examples in the standard, the number of benchmark tests, and the number of distinct language features exercised by those tests.

C11 Language Category	Examples	Tests	Features
§6.2 Concepts	3	1	1
§6.3 Conversions	0	15	3
§6.4 Lexical elements	11	1	1
§6.5 Expressions	23	44	19
§6.7 Declarations	44	51	14
§6.8 Statements	7	7	4
§6.9 External definitions	4	4	2
§6.10 Preprocessing directives	13	8	2
Total	**105**	**131**	**46**

B. Evaluation Metrics

For each test, a translation is considered correct if the generated Rust code compiles and, when executed, exhibits the same observable behavior as the original C program. Equivalence is determined by comparing observable behavior, including program output and exit status. The evaluation focuses on semantic translation correctness at the language feature level; we do not assess idiomatic quality or performance characteristics of the generated Rust code. All translations are required to use safe Rust only, and this is enforced at compile time, so every passing translation contains no compiler-detected use of an `unsafe` construct. A translation that fails to compile, panics at runtime, or diverges from the expected behavior is considered a failure. We define two metrics to characterize feature-level translation behavior.

a) Feature Coverage: Feature coverage measures whether a model successfully translates a given language feature. A feature is considered supported if the model produces a correct translation for at least one test exercising that feature.

b) Translation Consistency: Translation consistency measures how reliably a model translates a feature across benchmark instances. For a feature f and model m, translation consistency is defined as:

$$\mathrm{TC}(f, m) = \frac{\#\ \textit{tests of feature}\ f\ \textit{passed by}\ m}{\#\ \textit{tests of feature}\ f}. \quad (1)$$

III. Experimental Evaluation

We apply the benchmark to twelve large language models spanning several model families, sizes, and specializations. Including models from several families helps distinguish recurring failure patterns from those associated with one model or family. Our aim is to identify C language features that are difficult for LLM-based systems to translate, not to rank individual models.

The evaluated general-purpose models are GPT-4o, GPT-4 Turbo, GPT-4o-mini, Claude Opus 4.5, Claude Sonnet 4.5,

TABLE II: Aggregate translation success rate by model and temperature. Each entry is the mean fraction of correct translations across five independent samples per benchmark test.

Model	Temp = 0.2	Temp = 0.8
Small Models		
Gemma 3 12B Instruct	6.6%	5.6%
Llama 3.1 8B Instruct	6.0%	4.0%
Large General-Purpose Models		
Llama 3.1 70B	23.4%	24.0%
DeepSeek V3.2	34.8%	32.4%
Programming-Specialized Models		
Grok Code Fast 1	58.6%	58.3%
Mistral Codestral 2508	38.3%	36.0%
GPT-5.2-Codex	87.2%	86.7%
Proprietary General-Purpose Models		
GPT-4o	33.3%	31.6%
GPT-4 Turbo	47.3%	45.2%
Claude Opus 4.5	76.8%	76.6%
Claude Sonnet 4.5	48.2%	49.3%
Small Proprietary Model		
GPT-4o mini	6.9%	8.9%

Gemma 3 12B, Llama 3.1 (8B and 70B), and DeepSeek V3.2. The programming-specialized models are GPT-5.2-Codex, Grok-Code-Fast-1, and Mistral-Codestral-2508.

Each benchmark test is evaluated by prompting the model to translate the given C program into safe Rust. Translations are independent, without access to external tools or prior outputs. The prompt explicitly instructs the model to emit safe Rust only (no `unsafe`, raw pointers, or FFI), and this constraint is enforced at compile time via `RUSTFLAGS="-Funsafe_code"`, so any translation containing an `unsafe` construct fails to compile and is counted as a failure. A translation is considered successful only if it compiles under this safe Rust restriction, executes with the expected observable behavior, and is not a semantically incomplete "stub". The prompt, compilation and execution details, and stub detection are described in Appendix A.

A. Benchmark Calibration

Table II reports mean success rates per model, computed by drawing five independent samples per test, taking the fraction of those five that produce a correct translation, and averaging across all 131 tests. Mean success rates range from 4.0% to 87.2%, showing that model performance differs on the benchmark. Aggregate scores change little across the two decoding temperatures: no model shifts by more than 2.4 percentage points between temperatures 0.2 and 0.8. We therefore report subsequent results at temperature 0.2. The aggregate means obscure that no model handles the full benchmark: every model fails to produce any correct translation on at least 8 of the 131 tests, despite five sampling attempts. The feature-level analysis that follows identifies *which* C11 features drive this gap.

B. Analysis of Language Feature Support

We use the benchmark to characterize translation behavior at the level of individual C language features using the feature coverage and translation consistency metrics defined in Section II-B. Results in this section are computed under low-temperature decoding (temperature 0.2) with five independent samples per test.

a) Feature Coverage: Feature coverage measures whether a model translates a given language feature at all: a feature is covered by a model if at least one of its tests passes. Read under this binary criterion, a feature is covered by a model precisely when the corresponding cell in Table III is not orange. Coverage varies: **30 of 46 features** are covered by more than half of the evaluated models, while **16 of 46 features** are covered by six or fewer. One feature (`stmt.return`, the `return` statement) is covered by just two of twelve. These low-coverage rows identify features that are difficult for most evaluated models.

> For every feature in the benchmark, at least one model achieves coverage. Thus, at least one evaluated model produces a correct translation into safe Rust for one or more instances of every feature.

This result distinguishes feature-wide failure from failures confined to particular models or test instances; it does not establish the translatability of every benchmark instance.

b) Translation Consistency: Table III reports the translation consistency TC for every feature-model pair, shaded on a blue (high TC) to orange (low TC) scale. Beyond the binary covered or not-covered view, the cells with intermediate TC values reveal partial feature support: many features are correctly translated for some but not all of their exercising tests. Within a single clause, TC values for the same model can range from zero to one across adjacent features, indicating that the benchmark separates features that are uniformly easy or uniformly hard from features that are only intermittently translatable. Aggregating across models, no feature attains a translation consistency of one across all twelve: for every feature, at least one evaluated model fails at least one of its tests.

Of the **131 tests**, only **one** is passed by every model and **six** are failed by every model. The test passed by every model exercises C's standard compile-time idiom for computing the number of elements in an array via the `sizeof` operator: Rust provides this operation directly as the built-in `a.len()` method, so the translation is straightforward. The six tests failed by every model are instances for which none of the twelve evaluated models produces a translation into safe Rust. They sit within features that are otherwise covered: every feature has at least one test that some model translates correctly, but for these six tests no model provides a correct translation.

IV. Predictive Value of Feature Tests

To examine whether per-feature outcomes predict translation behavior on programs that contain multiple features, we construct a combined program whose runtime checks correspond one-to-one to the runtime-observable behaviors of three

	Programming-Specialized			Proprietary General-Purpose				Large General-Purpose		Small General-Purpose		
Feature	**Codex**	**Grok**	**Codestral**	**Opus**	**Sonnet**	**GPT-4T**	**GPT-4o**	**DeepSeek**	**L3-70B**	**Gemma 3**	**L3-8B**	**4o-mini**
Concepts (§6.2.4)												
Storage duration	1/1	1/1	0/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1	1/1	0/1
Conversions (§6.3)												
Bool/char/int conv.	3/3	3/3	3/3	2/3	3/3	1/3	0/3	3/3	1/3	0/3	0/3	0/3
Pointer conversions	6/6	6/6	1/6	5/6	4/6	3/6	3/6	3/6	1/6	0/6	0/6	0/6
Lvalue/array/function conv.	2/6	2/6	1/6	1/6	1/6	3/6	0/6	1/6	2/6	0/6	1/6	0/6
Lexical Elements (§6.4)												
Predefined identifiers	1/1	1/1	1/1	1/1	1/1	1/1	0/1	1/1	1/1	0/1	0/1	0/1
Expressions (§6.5)												
Bitwise shift	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	1/2	2/2	0/2
Multiplicative operators	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	0/1	0/1
Prefix inc/dec	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	0/2	0/2	1/2
Postfix inc/dec	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	0/1	0/1	0/1
Compound literals	8/8	6/8	5/8	7/8	4/8	7/8	7/8	8/8	6/8	3/8	3/8	5/8
Comma operator	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	1/2	0/2	0/2	0/2
Simple assignment	3/3	2/3	2/3	3/3	3/3	3/3	3/3	2/3	2/3	0/3	1/3	0/3
sizeof/alignof	2/3	2/3	2/3	2/3	2/3	2/3	2/3	2/3	1/3	1/3	2/3	2/3
Generic selection	1/1	1/1	1/1	1/1	1/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1
Additive operators	4/4	4/4	0/4	4/4	4/4	1/4	2/4	1/4	0/4	0/4	0/4	0/4
Logical AND	1/1	1/1	1/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1
Array subscripting	1/1	1/1	0/1	1/1	1/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1
Address & indirection	1/1	1/1	1/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1
Member access	4/4	3/4	1/4	4/4	1/4	1/4	1/4	1/4	2/4	1/4	0/4	0/4
Conditional operator	2/2	2/2	1/2	2/2	0/2	1/2	0/2	0/2	0/2	0/2	0/2	0/2
Function calls	1/1	1/1	0/1	1/1	0/1	0/1	0/1	1/1	0/1	0/1	0/1	0/1
Relational operators	4/4	4/4	1/4	3/4	1/4	0/4	1/4	1/4	1/4	0/4	0/4	0/4
Equality operators	2/2	2/2	0/2	2/2	0/2	0/2	0/2	1/2	0/2	0/2	0/2	0/2
Logical OR	1/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
Declarations (§6.7)												
Initialization	12/12	12/12	6/12	12/12	11/12	10/12	11/12	8/12	6/12	4/12	5/12	0/12
Type qualifiers	1/1	1/1	1/1	1/1	1/1	1/1	1/1	0/1	1/1	0/1	0/1	0/1
Tags	3/3	3/3	2/3	3/3	2/3	2/3	1/3	2/3	2/3	0/3	0/3	0/3
Type names	1/1	1/1	1/1	1/1	0/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1
Function declarators	5/5	5/5	4/5	5/5	1/5	3/5	1/5	2/5	0/5	0/5	0/5	0/5
Typedef	4/5	4/5	3/5	4/5	3/5	1/5	0/5	2/5	1/5	0/5	1/5	0/5
Structures and unions	5/5	3/5	1/5	5/5	1/5	4/5	1/5	1/5	0/5	0/5	0/5	2/5
Alignment specifiers	2/2	2/2	0/2	1/2	1/2	1/2	1/2	0/2	1/2	0/2	0/2	0/2
Function specifiers	2/3	2/3	1/3	1/3	1/3	1/3	2/3	1/3	1/3	0/3	1/3	0/3
Array declarators	4/4	4/4	1/4	3/4	0/4	1/4	0/4	1/4	1/4	0/4	0/4	0/4
Restrict qualifier	6/6	3/6	2/6	4/6	0/6	2/6	0/6	2/6	0/6	0/6	0/6	0/6
Enumeration	1/1	0/1	0/1	1/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
Pointer declarators	1/1	1/1	0/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
Atomic type specifiers	1/2	0/2	0/2	1/2	0/2	0/2	0/2	1/2	0/2	0/2	0/2	0/2
Statements (§6.8)												
Switch statement	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	1/1	0/1	1/1	0/1
Expression & null stmts	3/3	3/3	3/3	3/3	3/3	3/3	3/3	2/3	3/3	0/3	2/3	2/3
Goto statement	2/2	2/2	1/2	2/2	2/2	2/2	2/2	2/2	1/2	1/2	0/2	2/2
Return statement	1/1	0/1	0/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
External Definitions (§6.9)												
External object defs.	2/2	2/2	1/2	2/2	0/2	1/2	0/2	0/2	0/2	0/2	1/2	0/2
Function definitions	2/2	2/2	1/2	1/2	0/2	2/2	0/2	0/2	0/2	0/2	0/2	0/2
Preprocessing (§6.10)												
## operator	1/1	1/1	1/1	1/1	1/1	1/1	0/1	1/1	1/1	0/1	1/1	0/1
Macro scope	7/7	7/7	2/7	4/7	4/7	4/7	5/7	7/7	2/7	2/7	1/7	0/7

TABLE III: **Language feature support across the benchmark.** Each cell reports the translation consistency $\mathrm{TC}(f, m)$ for feature f and model m, displayed as the number of tests of f passed by m over the total tests of f. A test is counted as passed if at least one of five independent samples drawn at temperature 0.2 produces a correct, safe-Rust translation. A feature is considered supported by a model when $\mathrm{TC}(f, m) > 0$ (any non-orange cell). Color: ■ all tests pass to ■ no tests pass.

individual tests from the benchmark: multidimensional array indexing with pointer-arithmetic equivalence, rvalue member access on a struct returned by value, and by-value use of a typedef-named structure, which requires `Copy` in Rust. We evaluate the combined program at temperature 0.2 with five samples per test, matching Section III. The per-model mean success rates for the three individual tests and the combined program appear in Table IV (Appendix B-B). The results support three observations. First, all eight models with a zero mean success rate on at least one constituent test also have a zero mean success rate on the combined program. Second, all four models with nonzero mean success rates on all three constituent tests also have a nonzero mean success rate on the combined program. Third, we repeat the experiment with a program that performs the same checks but uses domain-specific identifier names (listed in Appendix B-A). Under the binary distinction between zero and nonzero success rates, the same eight models fail and the same four succeed. This limited comparison suggests that the result is not solely due to identifier naming.

V. Related Work

Prior work on C-to-Rust translation evaluates systems using whole-program benchmarks. `c2rust` evaluates translation success on a set of 11 C programs ranging in size from 27 to 201,695 lines of code, with a median project size of 776 lines, using compilation, testing, and execution cross-checking to validate semantic preservation. Follow-on systems such as Laertes [4] and Crown [7] focus on generating safe Rust and adopt similar evaluation methodologies, reusing the `c2rust` project-level benchmarks and extending it with additional programs. Laertes reports results on 17 programs with a median size of 5,002 lines of code, while Crown evaluates on 20 programs with a median size of 2,510 lines of code.

Unlike LLM-based translators, these static systems typically make explicit claims about their intended scope and limitations with respect to the C language. `c2rust` targets C99-compliant code and documents unsupported features such as variadic functions, `setjmp`/`longjmp`, inline assembly, `restrict` pointers, SIMD intrinsics without Rust equivalents, and certain floating-point extensions. Successor systems such as Laertes and Crown inherit these limitations and further refine the translation process, focusing on reducing specific classes of unsafety in the generated Rust rather than expanding the supported subset of C. Laertes and Crown operate as analysis- and rewrite-based extensions of `c2rust`, rather than as independent translators.

While these systems clearly articulate which C language features are supported, their evaluations do not systematically test that support at the level of individual language features. Feature support is instead inferred indirectly from whole-program success on a small set of benchmarks. This makes it difficult to determine whether a feature is handled correctly in general, or only in restricted or favorable usage patterns present in the evaluated programs.

The benchmark introduced in this paper is complementary to these existing evaluation practices. By isolating individual C language features and testing them using small, conformance-style programs derived from the C standard, our benchmark enables direct validation of claimed feature support. Applied to static C-to-Rust translators, it could be used to empirically assess conformance with the intended source-language subset (e.g., C99), distinguish robust feature support from partial or brittle implementations, and measure which language rules are exercised by existing benchmarks. While our experimental evaluation focuses on LLM-based translators, the benchmark itself can be applied to both static and learned translation systems.

Recent work on LLM-based C-to-Rust translation [8], [9], [10], [11], [12], [13], [14], [15], [16] adopts a similar project-level evaluation approach, emphasizing end-to-end translation quality and idiomatic target-language output. CRUST-Bench [21] provides a repository-scale benchmark with annotated Rust interfaces and correctness-enforcing test cases, enabling systematic end-to-end evaluation of LLM-based translators. While these studies demonstrate that LLMs can translate non-trivial programs, they provide limited evidence about systematic support for individual C language features. This motivates the use of feature-isolating, standard-derived benchmarks as a complementary evaluation method for both static and LLM-based translators.

VI. Conclusion and Future Work

This paper presented a feature-level validation framework for evaluating C-to-Rust code translation systems. The framework treats translation correctness as a property of individual language features rather than aggregate program success. It introduces a benchmark derived from the C11 standard that isolates core language features, together with feature coverage and translation consistency metrics for systematic assessment.

Applied to twelve large language models, the benchmark reveals feature-level failures under our evaluation criteria: even the highest-scoring model fails on 8 of 131 tests (Section III), and several C11 features are translated correctly by only a few models. Even features that are supported in aggregate exhibit instance-level variability that aggregate scores can obscure. These findings are consistent with traditional compiler testing, where success on many programs does not imply full language conformance. A combined-program case study (Section IV) provides preliminary evidence that per-feature test outcomes have predictive value for multi-feature program outcomes, suggesting that feature-level tests may be useful indicators of translation reliability.

This framework can be extended to assess additional dimensions of translation quality. We aim to combine the benchmark with program fuzzing and constrained LLM generation techniques [22] to generate larger and more diverse test suites for evaluating translation consistency and robustness.

References

[1] Immunant, Inc., "immunant/c2rust," https://github.com/immunant/c2rust, 2020, gitHub repository. Accessed 2025-11-04.

[2] Citrus Contributors, "Citrus," https://gitlab.com/citrus-rs/citrus, 2017, accessed 2017.
[3] Corrode Contributors, "Corrode," https://github.com/jameysharp/corrode, 2017, accessed 2017.
[4] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating C to safer Rust," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Oct. 2021. [Online]. Available: https://doi.org/10.1145/3485498
[5] J. Hong and S. Ryu, "Don't Write, but Return: Replacing Output Parameters with Algebraic Data Types in C-to-Rust Translation," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Jun 2024. [Online]. Available: https://doi.org/10.1145/3656406
[6] ——, "To Tag, or Not to Tag: Translating C's Unions to Rust's Tagged Unions," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 40–52. [Online]. Available: https://doi.org/10.1145/3691620.3694985
[7] H. Zhang, C. David, Y. Yu, and M. Wang, "Ownership Guided C to Rust Translation," in *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, 2023, pp. 459–482. [Online]. Available: https://doi.org/10.1007/978-3-031-37709-9_22
[8] V. Nitin, R. Krishna, L. Lemos do Valle, and B. Ray, "C2SaferRust: Transforming C Projects into Safer Rust with NeuroSymbolic Techniques," 2025. [Online]. Available: https://arxiv.org/abs/2501.14257
[9] A. Z. H. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, "Vert: Verified equivalent Rust transpilation with large language models as few-shot learners," 2024. [Online]. Available: https://arxiv.org/abs/2404.18852
[10] J. Hong and S. Ryu, "Type-migrating C-to-Rust translation using a large language model," *Empirical Software Engineering*, vol. 30, no. 1, p. 3, 2024. [Online]. Available: https://doi.org/10.1007/s10664-024-10573-2
[11] H. Ferit Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, "Towards translating real-world code with LLMs: a study of translating to Rust," 2025. [Online]. Available: https://arxiv.org/abs/2405.11514
[12] X. Cai, J. Liu, X. Huang, Y. Yu, H. Wu, C. Li, B. Wang, I. N. B. Yusuf, and L. Jiang, "RustMap: Towards Project-Scale C-to-Rust Migration via Program Analysis and LLM," in *Engineering of Complex Computer Systems: 29th International Conference, ICECCS 2025, Hangzhou, China, July 2-4, 2025, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 283–302. [Online]. Available: https://doi.org/10.1007/978-3-032-00828-2_16
[13] M. Shiraishi and T. Shinagawa, "Context-aware Code Segmentation for C-to-Rust Translation using Large Language Models," 2024. [Online]. Available: https://arxiv.org/abs/2409.10506
[14] Z. Tang, M. Agarwal, A. Shypula, B. Wang, D. Wijaya, J. Chen, and Y. Kim, "Explain-then-translate: an analysis on improving program translation with self-generated explanations," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 1741–1788. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.119/
[15] T. Zhou, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "LLM-driven multi-step translation from C to Rust using static analysis," in *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*, 2025. [Online]. Available: https://openreview.net/forum?id=i5MwgkeGZN
[16] M. Shetty, N. Jain, A. Godbole, S. A. Seshia, and K. Sen, "Syzygy: Dual Code-Test C to (safe) Rust Translation using LLMs and Dynamic Analysis," 2024. [Online]. Available: https://arxiv.org/abs/2412.14234
[17] Immunant, Inc., "C2Rust manual," https://c2rust.com/manual/, 2020, accessed 2025-11-04.
[18] T. H. Gibbs, B. A. Malloy, and J. F. Power, "Progression toward conformance of C++ language compilers," *Dr. Dobb's Journal*, vol. 28, no. 11, pp. 54–60, 2003.
[19] B. A. Malloy, S. A. Linde, E. B. Duffy, and J. F. Power, "Testing C++ compilers for ISO language conformance," *Dr. Dobb's Journal*, vol. 27, no. 6, pp. 71–78, 2002.
[20] Plum Hall Inc., "The Plum Hall Validation Suite," https://www.plumhall.com/newsite/stec1.html, accessed January 2026.
[21] A. Khatry, R. Zhang, J. Pan, Z. Wang, Q. Chen, G. Durrett, and I. Dillig, "CRUST-Bench: a comprehensive benchmark for C-to-safe-Rust transpilation," in *Second Conference on Language Modeling*, 2025. [Online]. Available: https://openreview.net/forum?id=8xofWL61S9
[22] E. Anaya Gonzalez, S. Vaidya, K. Park, R. Ji, T. Berg-Kirkpatrick, and L. D'Antoni, "Constrained sampling for language models should be easy: an MCMC perspective," *CoRR*, vol. abs/2506.05754, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2506.05754

APPENDIX A
EXPERIMENTAL DETAILS

This appendix documents the experimental setup used to evaluate C-to-Rust translations, including prompting, compilation and execution criteria, stub detection, and the execution environment.

A. Prompt Format

```
Convert the following C program to an
    equivalent standalone Rust program.

Rules:
- Preserve behavior exactly (same stdout and
    exit code).
- Must compile with rustc.
- Safe Rust only: no unsafe, raw pointers, or
    FFI.
- Use idiomatic Rust types/constructs where
    possible.

Output:
Return ONLY the Rust code in a single ```rust
    ``` code block, with a `fn main()` and no
    other text.
```

B. Compilation and Execution

Generated Rust programs are compiled using `rustc` with `RUSTFLAGS="-Funsafe_code"`, which causes the Rust compiler to reject any `unsafe` block, `unsafe fn`, or `unsafe impl` as a hard error. Any translation that passes compilation therefore contains no compiler-detected use of an `unsafe` construct. A translation is considered successful if the program compiles and, when executed, produces the expected observable behavior of the original C program, as defined in section II-B.

Details of the compilation toolchains and execution platform used in the experiments described in this paper are provided in Section A-D.

C. Stub Detection

In addition to compilation and execution checks, we apply a stub-detection pass to filter translations that are syntactically valid but semantically incomplete. The detector compares each generated Rust program against a structural baseline derived from static analysis of the original C code, capturing properties such as program size, function definitions, control flow, and nontrivial function calls. It also checks for common stub patterns, including literal-only returns, print-only programs, and explicit placeholder markers, and verifies symbol coverage when available. Translations flagged as stubs are treated as failed attempts and are not counted as successful translations, even if they compile and run.
```

D. Environment Details

All experiments were conducted on macOS 15.7.3 running on an arm64 system (Apple M4 Max). C programs were compiled using Apple Clang 17.0.0 via the `gcc` driver, and generated Rust programs were compiled and executed using `rustc` 1.91.0-nightly (7ad23f43a, 2025-09-09).

APPENDIX B
ADDITIONAL EXPERIMENTAL EVALUATION

This appendix provides additional experimental results extending the main evaluation in Section III, including a detailed breakdown of aggregate outcomes by compilation and stub detection.

A. Combined Program Listing: Pathfinder

Figure 2 shows `pathfinder.c`, the combined program described in Section IV that uses names with domain-specific meaning: the structs are named `Coord` and `Waypoint`, the helper functions are named `make_coord`, `shift_row`, and `entry_cost`, and the local variables are named `start`, `goal`, and `cursor`. The other combined program, `combined_features.c`, performs the same runtime checks but uses generic names with no such meaning. Differences between results on the two programs may therefore indicate sensitivity to identifier naming.

B. Per-Model Proxy Table

Table IV reports per-test mean success rate at temperature 0.2 (five samples per test) for the three individual tests incorporated into `combined_features.c` and for the two combined programs discussed in Section IV.

TABLE IV: Per-test mean success rate at temperature 0.2 (five samples per test). "Indiv." columns are the three individual tests incorporated into the combined programs; "Combined" columns are `combined_features.c` (feat.) and `pathfinder.c` (path.).

	Indiv. tests			Combined	
Model	**array**	**member**	**typedef**	**feat.**	**path.**
Small Models					
Gemma 3 12B Instruct	0%	100%	0%	0%	0%
Llama 3.1 8B Instruct	0%	20%	0%	0%	0%
Large General-Purpose Models					
Llama 3.1 70B	0%	100%	0%	0%	0%
DeepSeek V3.2	0%	100%	0%	0%	0%
Programming-Specialized Models					
Grok Code Fast 1	40%	100%	40%	60%	40%
Mistral Codestral 2508	0%	100%	0%	0%	0%
GPT-5.2-Codex	100%	100%	100%	100%	100%
Proprietary General-Purpose Models					
GPT-4o	0%	100%	0%	0%	0%
GPT-4 Turbo	20%	100%	0%	0%	0%
Claude Opus 4.5	80%	100%	100%	100%	100%
Claude Sonnet 4.5	40%	100%	100%	20%	60%
Small Proprietary Model					
GPT-4o mini	0%	0%	0%	0%	0%

```c
/* Combines three features:
   1: expr.postfix.subscript
   2: expr.postfix.member
   3: decl.type_defs */
#include <stddef.h>
#include <stdio.h>

typedef struct coord    { int row; int col; } Coord,
     *CoordPtr;
typedef struct waypoint { int cost; }         Waypoint,
     *WaypointPtr;

static Coord make_coord(int r, int c){ Coord v={r,c};
     return v; }
static int coord_sum  (Coord v)        { return
     v.row+v.col+1; }
static int coord_sum_s(struct coord v) { return
     v.row+v.col+2; }
static int shift_row  (CoordPtr p)     { p->row+=100;
     return p->row; }
static int padded     (int v)          { return v+4; }
static int entry_cost (Waypoint v)     { return v.cost+5; }

int main(void) {
  /* feature 1: subscript fill + pointer-arithmetic
       equivalence */
  int x[3][5];
  for (int i=0;i<3;++i) for (int j=0;j<5;++j) x[i][j]=i+j;
  for (int i=0;i<3;++i) for (int j=0;j<5;++j)
    if (x[i][j] != *(*(x+i)+j)) return 1;
  int *p0=&x[0][0], *p1=&x[1][0];
  if ((ptrdiff_t)(p1-p0) != 5) return 2;

  /* feature 2: rvalue member access on returned struct */
  if (make_coord(x[1][0],x[1][1]).row != x[1][0]) return 3;
  if (make_coord(x[1][0],x[1][1]).col != x[1][1]) return 4;
  if (make_coord(x[0][3],x[1][2]).row
    + make_coord(x[2][0],x[0][4]).col != 7) return 5;

  /* feature 3: same var passed to typedef- and
       struct-tag-named params */
  Coord start; struct coord goal; CoordPtr cursor;
  start.row=x[1][0]; start.col=x[1][1];
  goal.row =x[2][0]; goal.col =x[2][1];
  if (coord_sum  (start) != 4) return 6;
  if (coord_sum_s(start) != 5) return 7;
  if (coord_sum  (goal)  != 6) return 8;
  if (coord_sum_s(goal)  != 7) return 9;

  cursor = &start;
  if (shift_row(cursor) != 101) return 10;
  if (start.row         != 101) return 11;
  if (padded(start.row) != 105) return 12;

  int q = (int)sizeof(Coord) + (int)sizeof(struct coord);
  if (q == 0) return 13;

  Waypoint wp; WaypointPtr wp_ptr;
  wp.cost = x[2][0]; wp_ptr = &wp;
  if (wp_ptr->cost != x[2][0]) return 14;
  if (entry_cost(wp) != 7)     return 15;

  printf("PASS\n");
  return 0;
}
```

Fig. 2: `pathfinder.c`: combined program with domain-specific naming.

C. Outcome Decomposition and Stub Detection

This section extends the aggregate performance analysis in Section III-A by decomposing the per-model translation success rate from Table II into three constituent rates: the rate at which generated Rust code compiles under the safe Rust restriction (**CR**); the rate at which translations are classified

TABLE V: Per-model decomposition of translation outcomes at temperatures 0.2 and 0.8. For each (model, temperature) cell we report the mean success rate (**SR**), the mean compile rate (**CR**, fraction of samples that produce compiling Rust), and the mean stub-detection rate (**SD**, fraction of samples flagged as semantically incomplete stubs). All three metrics are computed per test as the fraction of five independent samples satisfying the criterion, then averaged across the 131 benchmark tests.

	Temp = 0.2			**Temp = 0.8**		
Model	**SR**	**CR**	**SD**	**SR**	**CR**	**SD**
Small Models						
Gemma 3 12B Instruct	6.6%	9.6%	8.4%	5.6%	8.5%	7.8%
Llama 3.1 8B Instruct	6.0%	11.3%	2.4%	4.0%	7.9%	3.1%
Large General-Purpose Models						
Llama 3.1 70B	23.4%	29.0%	3.7%	24.0%	30.1%	3.8%
DeepSeek V3.2	34.8%	40.9%	1.4%	32.4%	38.9%	1.4%
Programming-Specialized Models						
Grok Code Fast 1	58.6%	66.4%	2.3%	58.3%	67.0%	2.7%
Mistral Codestral 2508	38.3%	51.0%	1.4%	36.0%	46.9%	1.8%
GPT-5.2-Codex	87.2%	93.7%	1.2%	86.7%	94.0%	1.4%
Proprietary General-Purpose Models						
GPT-4o	33.3%	41.1%	0.5%	31.6%	37.3%	0.5%
GPT-4 Turbo	47.3%	56.5%	2.7%	45.2%	55.3%	3.5%
Claude Opus 4.5	76.8%	91.3%	5.8%	76.6%	91.0%	5.8%
Claude Sonnet 4.5	48.2%	55.3%	1.2%	49.3%	55.1%	0.6%
Small Proprietary Model						
GPT-4o mini	6.9%	9.6%	0.2%	8.9%	11.5%	0.8%

as syntactically valid but semantically incomplete stubs (**SD**, per the criteria in Section A-C); and the overall success rate (**SR**), reproduced here for direct comparison. Together these decompose unsuccessful translations into failures due to non-compiling Rust, stub outputs, and remaining semantic mismatches at execution time. Results at both temperatures evaluated in Table II are reported in Table V.